8 Multithreading

8.1 Thread

It is the basic unit of program execution. A process can have several threads running concurrently, each performing a different job. When a thread has finished its job, it is suspended or destroyed.

Threading distinguishes Java from C++, and is the ability to have multiple "simultaneous" lines of execution running in a single program. If you are still stuck with a computer that has only one CPU, it's a myth that separate threads run simultaneously. Because they are sharing the cycles of the one CPU, each thread is given a priority – which is a number from 1-10. (5 is the default priority.) Threads with a higher priority number are given more turns at the CPU. In fact, depending on the operating system, a thread's priority can have a more drastic impact.

Depending on the OS, one of either two strategies is used:

- preemption or
- timeslicing.

Under Sun's Solaris operating system, a thread of a certain priority is allowed to run until it is finished, or until a thread of higher priority becomes ready to run.

When a thread of higher priority arrives and steals the processor from a lower-priority thread, this is known as preemption. One weakness of the preemption system is this: if another thread of the same priority is waiting to run, it is shut out of the processor. This is called thread starvation.

Another system is used in the Windows world. In a process known as timeslicing, each thread is given a period of time to run, and then all the other threads of equal priority get their turn at the CPU. As with the preemption system, under timeslicing, a higher-priority thread still commandeers the CPU immediately.

8.2 Class Thread: an Overview

`java.lang.Thread`

When we wish to make a Java program use threading, the easiest way is to say our class extends the class Thread. The Java Class Thread has 7 different constructors.

1.Thread()—this constructor takes no arguments.

Every instantiated thread object has a name. When we use the no-argument